



CSE 3313: Computer Architecture

Sidratul Tanzila Tasmi

Lecturer, Department of Computer Science and Engineering
United International University

Tag Bit and Valid Bit



The tag in a cache is used to identify which specific block of memory is currently stored in a particular cache line. It's essential for determining whether the requested memory data is already in the cache (cache hit) or not (cache miss).

valid bit - A bit of information that indicates whether the data in a block is valid (1) or not (0).

Locating Data in Cache



Given an address, we can determine whether the data at that memory location is in the cache. To do so, we use the following procedure:

1. Use the set index to determine which cache set the address should reside in.
2. For each block in the corresponding cache set, compare the tag associated with that block to the tag from the memory address. If there is a match, proceed to the next step. Otherwise, the data is not in the cache.
3. For the block where the data was found, look at valid bit. If it is 1, the data is in the cache, otherwise it is not.

Actual Cache Size



Cache Line Structure

Each cache block (line) stores:

Tag (from memory address)

Valid Bit

Data Block

Solved Problem



Suppose you are required to design a 512 KB cache with 4 words data per block. Each word contains 4 bytes of data. There are 32 bits the memory address.

- (a) Calculate the number of lines in the cache
- (b) Find out the number of index bits and tag bits
- (c) Calculate the actual size of the cache

Solved Problem



a.
Cache Size = 512 KB = 512×1024 bytes = 524,288 bytes
Block size = 4 words/block
Word size = 4 bytes
So, block size in bytes = $4 \times 4 = 16$ bytes

Number of lines = Cache size / block size = $524,288 / 16$
= 32,768 blocks

Solved Problem



Let's first break down a 32-bit address for a direct-mapped cache.

Offset bits (block offset): Number of bits to address a byte within a block

Offset bits= $\log_2(16)=4$ bits

Offset bits= $\log_2(16)=4$ bits

Index bits: Number of bits to index the block in the cache

Index bits= $\log_2(32768)=15$ bits

Tag bits: Remaining bits from 32

Tag bits= $32-(15+4)=13$ bits

Tag bits= $32-(15+4)=13$ bits

Solved Problem



Actual size includes data + metadata (tag + valid bit).

Data size = 524,288 bytes (already given)

For each block, we store: Tag = 13 bit, Valid bit = 1 bit

So, Metadata per block = 14 bits

Total blocks = 32,768

Total metadata size = $32,768 \times 14 = 458,752$ bits = 57,344 bytes

So, actual cache size = Data (524,288 bytes) + Metadata (57,344 bytes)

= 581,632 bytes

≈ 568 KB

Cache Read Algorithm



Look at Processor Address, search cache tags to find match. Then either

Found in cache
a.k.a. HIT

Not in cache
a.k.a. MISS

Return copy
of data from
cache

Read block of data from
Main Memory

Wait ...

Return data to processor
and update cache

Cache Example



- Cache 8-blocks, Ram 32-Blocks, 1 word/block, direct mapped
- Initial state

Index	V	Tag	Data
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	N		
111	N		

Cache Example



Word addr	Binary addr	Hit/miss	Cache block
22	10 110	Miss	110

Index	V	Tag	Data
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	Y	10	Mem[10110]
111	N		

Index	V	Tag	Data
000	N		
001	N		
010	Y	11	Mem[11010]
011	N		
100	N		
101	N		
110	Y	10	Mem[10110]
111	N		

Cache Example

Word addr	Binary addr	Hit/miss	Cache block
22	10 110	Hit	110
26	11 010	Hit	010

Index	V	Tag	Data
000	N		
001	N		
010	Y	11	Mem[11010]
011	N		
100	N		
101	N		
110	Y	10	Mem[10110]
111	N		

Cache Example



Word addr	Binary addr	Hit/miss	Cache block
16	10 000	Miss	000
3	00 011	Miss	011
16	10 000	Hit	000

Index	V	Tag	Data
000	Y	10	Mem[10000]
001	N		
010	Y	11	Mem[11010]
011	Y	00	Mem[00011]
100	N		
101	N		
110	Y	10	Mem[10110]
111	N		

Cache Example



Word addr	Binary addr	Hit/miss	Cache block
18	10 010	?	010

Index	V	Tag	Data
000	Y	10	Mem[10000]
001	N		
010	Y	11	Mem[11010]
011	Y	00	Mem[00011]
100	N		
101	N		
110	Y	10	Mem[10110]
111	N		

Index	V	Tag	Data
000	Y	10	Mem[10000]
001	N		
010	Y	10	Mem[10010]
011	Y	00	Mem[00011]
100	N		
101	N		
110	Y	10	Mem[10110]
111	N		

Cache Example



Physical Memory Address = 10 bit

Cache Memory Size = 8 words

Block size = 4 words

- a) Calculate the number of blocks
- b) Calculate the number of lines
- c) Find the size of index field for block
- d) Find the size of offset field

Cache Example



Physical Memory Address = 10 bit

Cache Memory Size = 8 words

Block size = 4 words

a) Calculate the number of blocks

$$\begin{aligned}\text{No of blocks} &= \text{Physical memory size} / \text{block size} \\ &= 2^{10} / (4 \times 4 \times 8) = 8\end{aligned}$$

Cache Example



Physical Memory Address = 10 bit

Cache Memory Size = 8 words

Block size = 4 words

- a) Calculate the number of blocks
- b) Calculate the number of lines

Number of lines = cache memory size / block size
 $= 8 / 4 = 2$

Cache Example



Physical Memory Address = 10 bit

Cache Memory Size = 8 words

Block size = 4 words

- a) Find the size of index field for block
- b) Find the size of offset field

Index field for block = $\log_2 (\text{no of blocks}) = \log_2 (8) = 3$

Offset = physical memory address - index field size = $10 - 3 = 7$

Cache Example 2



Physical Memory size = 6 bit

Size of Index field = 4 bits

Number of lines in cache = 4

- a) Find the number of bits in block offset
- b) Calculate the size of the blocks

Cache Example 2



Physical Memory size = 6 bit

Size of Index field = 4 bits

Number of lines in cache = 4

- a) Find the number of bits in block offset
Physical memory size - index field size = $6 - 4 = 2$
- b) Calculate the size of the blocks

Cache Example 2



Physical Memory size = 6 bit

Size of Index field = 4 bits

Number of lines in cache = 4

- a) Find the number of bits in block offset
Physical memory size - index field size = $6 - 4 = 2$
- b) Calculate the size of the blocks - $2^2 = 4$

Cache Example 3



1	0	1	0	1	1
---	---	---	---	---	---

Physical Memory size = 6 bit

Size of Index field = 4 bits

Number of lines in cache = 4

- a) Calculate which line the following binary number belong to? (in question, they might give you directly a word value in decimal, convert it into binary)

Cache Example 3 – Approach 1



1	0	1	0	1	1
---	---	---	---	---	---

Physical Memory size = 6 bit

Size of Index field = 4 bits

Number of lines in cache = 4

- a) Calculate which line the following binary number belong to? (in question, they might give you directly a word value in decimal, convert it into binary)

Block Number bit requirement = index field size = 4

1010 = Block 10 - > Line Number = $10 \% 4$ (block number % total lines) = 2

Cache Example 3 – Approach 2



1	0	1	0	1	1
---	---	---	---	---	---

Physical Memory size = 6 bit

Size of Index field = 4 bits

Number of lines in cache = 4

- a) Calculate which line the following binary number belong to? (in question, they might give you directly a word value in decimal, convert it into binary)

From the 4 bits used to determine block number, some bits will be used to determine line. Bits required to represent line number = $\log_2$ (no of lines) = $\log_2 (4) = 2$

Cache Example 3 - Approach 2



1	0	1	0	1	1
---	---	---	---	---	---

Physical Memory size = 6 bit

Size of Index field = 4 bits

Number of lines in cache = 4

- a) Calculate which line the following binary number belong to? (in question, they might give you directly a word value in decimal, convert it into binary)

From the 4 bits used to determine block number, some bits will be used to determine line. Bits required to represent line number = $\log_2$ (no of lines) = $\log_2 (4) = 2$

The green portion shows line number = $10 = 2$

Cache Example 3 - Approach 2



1	0	1	0	1	1
---	---	---	---	---	---

Physical Memory size = 6 bit

Size of Index field = 4 bits

Number of lines in cache = 4

Index field - bits required to represent line number = Tag bits

Use of Tag Bits



1	0	1	0	1	1
---	---	---	---	---	---

Check the hit and miss ratio if the numbers are inserted in the cache sequentially.

- 1) 101011 - Miss Not found in cache

Line	Meta Data Tag Bits + 1 Valid bit	Actual Data
L0		
L1		
L2		
L3		

Use of Tag Bits



1	0	1	1	0	1
---	---	---	---	---	---

Check the hit and miss ratio if the numbers are inserted in the cache sequentially.

- 1) 101011 - Miss Not found in cache
- 2) 101101 - Miss

Line	Meta Data Tag Bits + 1 Valid bit	Actual Data
L0		
L1		
L2	10	B10
L3	10	B11

Use of Tag Bits



1	1	1	1	1	0
---	---	---	---	---	---

Check the hit and miss ratio if the numbers are inserted in the cache sequentially.

- 1) 101011 - Miss Not found in cache
- 2) 101101 - Miss
- 3) 111110 - Miss

Line	Meta Data Tag Bits + 1 Valid bit	Actual Data
L0		
L1		
L2	10	B10
L3	10	B11

Use of Tag Bits



1	1	1	1	0	1
---	---	---	---	---	---

Check the hit and miss ratio if the numbers are inserted in the cache sequentially.

- 1) 101011 - Miss Not found in cache
- 2) 101101 - Miss
- 3) 111110 - Miss
- 4) 111101 - Hit

Line	Meta Data Tag Bits + 1 Valid bit	Actual Data
L0		
L1		
L2	10	B10
L3	11	B15

- 1) Tag bits
- 2) 1 Valid Bit

So information in each line:
cache data + meta data

Line	Meta Data Tag Bits + 1 Valid bit	Actual Data
L0		
L1		
L2	10	B10
L3	11	B15

Formula to calculate size of cache



Number of Lines x (1 valid bit + Number of Tag bits + Size of cache [given in question or you have to find it, but you must convert it into bits])

Line	Meta Data Tag Bits + 1 Valid bit	Actual Data
L0		
L1		
L2	10	B10
L3	11	B15

Block Size Considerations



- Larger blocks should reduce miss rate
 - Due to spatial locality
- But in a fixed-sized cache
 - Larger blocks $\Rightarrow$ fewer of them
 - More competition $\Rightarrow$ increased miss rate
- Larger miss penalty
 - Can override benefit of reduced miss rate

Cache Size



- 32-bit byte address (but byte address might be different in your question!)
 - A direct-mapped cache
 - The cache size is 2^n blocks
 - The block size is 2^m words (2^{m+2} bytes)
 - What is the size of the tag field?
 - $32 - (n + m + 2)$
- | | |
|-----|---------------------------|
| n | select cache block |
| m | select words within block |
| 2 | select byte within word |

Cache Size

- 32-bit byte address
- A direct-mapped cache
- The cache size is 2^n blocks
- The block size is 2^m words (2^{m+2} bytes)
- What is the size of the cache (unit: no. of bit)?
 - $2^n \times (2^m \times 32 + 32 - (n+m+2) + 1)$

Mapping an Address to a Multi Word Cache Block

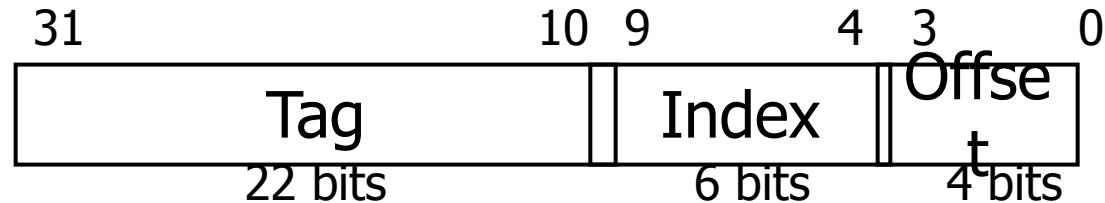


64 blocks, 16 bytes/block

To what block number does address 1200 map?

Block address = $\lfloor 1200/16 \rfloor = 75$

Block number = $75 \text{ modulo } 64 = 11$



Handling Cache Miss



Read misses

Stall execution, fetch the block from the next level in the memory hierarchy, install it in the cache, send requested word to processor, and then let execution resume

Write misses

Write allocate: Stall execution, fetch the block from next level in the memory hierarchy, install it in cache, write the word from processor to cache, also update memory, then let execution resume

or

No-write allocate: skip the cache write and just write the word to memory



Thank you!